R Lab4. Relacionando enfermedad y genética

Bioinformática. Ingeniería Biomédica

¿Qué posiciones del DNA se pueden relacionar con una enfermedad?

¿Cuándo decimos que una enfermedad es genética?

- Se ha encontrado diferencia significativa entre la información genética de los individuos que tienen la enfermedad y los sanos
- ¿Cuánta diferencia?
- ¿Exactamente qué información genética?

Estudios de asociación del genoma completo

Los estudios de asociación del genoma completo (GWAS, *Genome Wide Association Studies*) son un método para identificar *locus* responsables de otorgar cierta susceptibilidad a desarrollar enfermedades complejas o multifactoriales, aquellas causadas por la interacción de múltiples genes y factores ambientales.

Se basan en mapear el genoma de muchos individuos para identificar la variación genética posiblemente responsable de una enfermedad utilizando técnicas estadísticas. En general, se necesitarán un gran número de sujetos para poder identificar de forma fiable esta variación.

La tecnología de microarrays para la detección de polimorfismos de un solo nucleótido (SNP), y el proyecto *HapMap* junto con el proyecto Genoma Humano, han hecho posible llevar a cabo este tipo de estudios.

Una base de datos sobre SNPs es **dbSNP** disponible a través de NCBI (https://www.ncbi.nlm.nih.gov/snp/). Otra fuente importante de información sobre fenotípicos genéticos es OMIM (*Online Mendelian Inheritence in Man*) (https://omim.org/).

Objetivo

Identificar variantes genéticas asociadas con un fenotipo de enfermedad. Simplificando el problema, identificar la asociación entre un fenotipo (normalmente tiene la enfermedad) con uno o varios SNPs. Para ello:

- Se usan dos grupos de individuos: los controles (sin enfermedad) y los casos (con la enfermedad).
- Se miden los SNPs en todos los individuos.
- Se evalúa la asociación entre ser caso / control y la presencia de cada uno de los SNPs.

Asociación de un SNP

El análisis estadístico de los datos GWAS tratan de establecer la asociación entre cada SNP individual y el fenotipo observado. El fenotipo suele ser una enfermedad, la respuesta a un fármaco, u otros fenotipos.

Vamos a utilizar el paquete `SNPassoc` disponible en CRAN para llevar a cabo este análisis.

0.1 Instala y carga los paquetes que vamos a utilizar.

Solución: Debéis tener en cuenta que `SNPassoc` está disponible en CRAN y el resto en Bioconductor.

```
if(!("SNPassoc" %in% installed.packages()))
  install.packages("SNPassoc")
library("SNPassoc")
```

Vamos a utilizar un conjunto de datos disponible en este paquete, denominado `SNPs` que cargamos con la función `data()`. Este conjunto de datos contiene la información 35 SNPs en 157 sujetos. Además contiene las siguientes variables fenotípicas:

- `casco` (1 si es caso y 0 si control),
- el sexo (`sex`),
- la presión arterial (`blood.pre`) y
- los niveles de cierta proteína (`protein`).

0.1 Carga los datos con la función `data()`.

Solución:

```
data(SNPs, package = "SNPassoc")
str(SNPs, list.len=6)
```

```
## 'data.frame':    157 obs. of  40 variables:
##  $ id       : int  1 2 3 4 5 6 7 8 9 10 ...
##  $ casco    : num  1 1 1 1 1 1 1 1 1 1 ...
##  $ sex      : Factor w/ 2 levels "Male","Female": 2 2 2 1 2 2 2 1 1 2 ...
##  $ blood.pre: num  13.7 12.7 12.9 14.6 13.4 11.3 11.9 12.4 14.5 12.2 ...
##  $ protein  : num  75641 28688 17280 27254 38067 ...
##  $ snp10001 : Factor w/ 3 levels "CC","CT","TT": 3 3 3 2 3 3 3 3 2 3 ...
##   [list output truncated]
```

---

1. Debemos transformar el `data.frame` con los datos a un objeto de la clase `snp`. Para ello podemos utilizar la función `setupSNP()` del paquete `SNPassoc` cuya sintaxis básica es:

```
setupSNP(data, colSNPs, sep = "/", ...)
```

Debemos indicarle el `data.frame` que contiene los datos (`data`), las columnas de este `data.frame` en las que se encuentra la información sobre los SNPs con el argumento `colSNPs` y también el caracter separador para dividir los alelos en los genotipos (`sep`). Este tipo de objeto tiene una estructura similar a un `data.frame`, ligeramente diferente en la forma en la que se codifican los SNPs. Transforma el `data.frame` `SNPs` en un objeto de la clase `snp`. En este caso, el caracter separador para dividir los alelos en los genotipos es el vacío.

Solución:

```
mySNP <- setupSNP(SNPs, colSNPs=6:40, sep="")
head(mySNP, n=3)
```

```
##   id casco    sex blood.pre  protein snp10001 snp10002 snp10003 snp10004
## 1  1     1 Female      13.7 75640.52      T/T      C/C      G/G      G/G
## 2  2     1 Female      12.7 28688.22      T/T      A/C      G/G      G/G
## 3  3     1 Female      12.9 17279.59      T/T      C/C      G/G      G/G
##   snp10005 snp10006 snp10007 snp10008 snp10009 snp100010 snp100011 snp100012
## 1      G/G      A/A      C/C      C/C      A/A       T/T       G/G       G/G
## 2      A/G      A/A      C/C      C/C      A/G       T/T       G/G       C/G
## 3      G/G      A/A      C/C      C/C      A/A       T/T       C/C       G/G
##   snp100013 snp100014 snp100015 snp100016 snp100017 snp100018 snp100019
## 1       A/A       A/A       G/G       G/G       T/T       T/T       C/C
## 2       A/A       A/C       G/G       G/G       C/T       C/T       C/G
## 3       A/A       C/C       G/G       G/G       T/T       T/T       C/C
##   snp100020 snp100021 snp100022 snp100023 snp100024 snp100025 snp100026
## 1       G/G       G/G       A/A       T/T       T/T       C/C       G/G
## 2       G/G       G/G       A/A       A/T       T/T       C/C       G/G
## 3       G/G       G/G       A/A       T/T       T/T       C/C       G/G
##   snp100027 snp100028 snp100029 snp100030 snp100031 snp100032 snp100033
## 1       C/C       C/C       G/G       A/A       T/T       A/A       A/A
## 2       C/G       C/T       G/G       A/A       T/T       A/G       A/G
## 3       C/C       C/C       G/G       A/A       T/T       A/A       A/A
##   snp100034 snp100035
## 1       T/T       T/T
## 2       T/T       T/T
## 3       T/T       T/T
```

---

2. Vamos a contrastar la asociaciación de un SNP, el `snp10001`, con ser caso o control utilizando la función `SNPassoc::association()`.

- El primer argumento de esta función es una fórmula que relaciona la variable dependiente, `casco`, con el SNP evaluado `snp10001` a través del operador `~`.
- Es posible añadir otras variables utilizando el operador `+`. En este caso, ajustamos el resultado utilizando las variables `sex` y `blood.pre`.

La función `association()` puede modelizar la dependencia según modelos de herencia codominante, dominante, recesivo, overdominante y log-aditivo. Indicamos esta información utilizando el argumento `model`. Interpreta la salida que obtienes para los modelos dominante y codominante.

Solución:

```
myres <- association(casco~sex+snp10001+blood.pre,
                     data = mySNP,
                     model = c("dominant","codominant"))
myres
```

```
##
## SNP: snp10001  adjusted by: sex blood.pre
##              0    %  1    %   OR lower upper p-value   AIC
## Codominant
## T/T        24 51.1 68 61.8 1.00              0.1541 195.8
## C/T        21 44.7 32 29.1 0.55  0.26  1.14
## C/C         2  4.3 10  9.1 1.74  0.35  8.63
## Dominant
## T/T        24 51.1 68 61.8 1.00              0.2286 196.1
## C/T-C/C    23 48.9 42 38.2 0.65  0.32  1.31
```

El resultado contiene información sobre los tamaños muestrales, las frecuencias de los genotipos, las odds-ratio (OR) y sus intervalos de confianza al 95%, el p-valor asociado al contraste de asociación y el criterio de información de Akaike (AIC) de cada modelo genético. Notad que la OR toma como categoría de referencia el genotipo homocigoto más frecuente.

Asociación de múltiples SNPs

El objetivo es evaluar la asociación de múltiples SNPs, en todos los SNP disponibles en los datos.

1. Utilizamos la función `WGassociation()` para evaluar la asociación de todos los SNPs disponibles en el objeto la clase `snp` creado en la sección anterior.

- En el primer argumento indicamos la **variable dependiente** y podemos indicarle una fórmula para ajustar los resultados por variables fenotípicas, sin incluir los SNPs.
- Con el argumento `model` indicamos el o los modelos de herencia que queremos evaluar. Con `model="all"` indicamos todos los modelos.

Realiza de los contrastes con todos los SNPs y todos los modelos utilizando como variables confusoras el sexo (`sex`) y la presión sanguínea (`blood.pre`).

Solución:

```
myres <- WGassociation(casco~sex+blood.pre, data = mySNP, model = "all")
myres
```

```
##           comments    codominant dominant recessive overdominant log-additive
## snp10001  -           0.15410    0.22859  0.28494   0.07188      0.60861
## snp10002  -           0.16762    0.93397  0.06177   0.55418      0.69503
## snp10003  Monomorphic -          -        -         -            -
## snp10004  Monomorphic -          -        -         -            -
## snp10005  -           0.36192    0.89835  0.16153   0.67485      0.87744
## snp10006  Monomorphic -          -        -         -            -
## snp10007  Monomorphic -          -        -         -            -
## snp10008  -           0.60118    0.62090  0.51107   0.39203      0.88957
## snp10009  -           0.17040    0.92127  0.06331   0.54568      0.70775
## snp100010 Monomorphic -          -        -         -            -
## snp100011 -           0.56047    0.87900  0.38672   0.51851      0.86165
## snp100012 -           0.39422    0.88410  0.17243   0.88969      0.68687
## snp100013 -           0.37134    0.41316  0.45343   0.19849      0.73794
## snp100014 -           0.54059    0.49343  0.56075   0.26941      0.88444
## snp100015 -           0.80736    -        -         -            -
## snp100016 Monomorphic -          -        -         -            -
## snp100017 -           0.17683    0.86588  0.06789   0.50913      0.76452
## snp100018 -           0.16090    0.79720  0.06331   0.44837      0.81799
## snp100019 -           0.27982    0.25172  0.53644   0.11222      0.68788
## snp100020 -           0.54132    0.54211  0.51107   0.32683      0.81998
## snp100021 Monomorphic -          -        -         -            -
## snp100022 Monomorphic -          -        -         -            -
## snp100023 -           0.16568    0.92793  0.06113   0.54630      0.69761
## snp100024 -           0.32111    0.32077  0.47147   0.14650      0.65751
## snp100025 Monomorphic -          -        -         -            -
## snp100026 Monomorphic -          -        -         -            -
## snp100027 -           0.14315    0.69405  0.06044   0.36716      0.90989
## snp100028 -           0.23015    0.25914  0.44730   0.08682      0.77026
## snp100029 -           0.52008    0.57163  0.47147   0.30721      0.90555
## snp100030 Monomorphic -          -        -         -            -
## snp100031 Genot error -          -        -         -            -
## snp100032 -           0.31362    0.35900  0.44767   0.12798      0.87334
## snp100033 -           0.25908    0.33676  0.39283   0.10057      0.90111
## snp100034 -           0.52008    0.57163  0.47147   0.30721      0.90555
## snp100035 Monomorphic -          -        -         -            -
```

Notad que la información que se obtiene es una tabla resumen con los p-valores asociados a cada SNP/modelo de herencia. El carácter `-` indica que no es posible ajustar ese modelo porque no hay datos suficientes.

---

2. Para obtener los resultados detallados de un SNP concreto se puede utilizar la función `WGstats()` pasándole como argumento el objeto obtenido como resultado de la función `WGassociation()`. ¿De que clase es el objeto que resulta de la función `WGstats()`? Obtén la información detallada del análisis para el primer SNP.

Solución:

```
myres.list <- WGstats(myres)
class(myres.list)
myres.list[[1]]
```

La función `WGstats()` devuelve una lista en la que cada elemento contiene la información de cada SNP evaluado.

---

3. La variable dependiente puede ser cualitativa o cuantitativa. Evalúa la asociación con la variable `protein`.

Solución:

```
myres2 <- WGassociation(protein~sex+blood.pre, data = mySNP, model = "all")
myres2
```

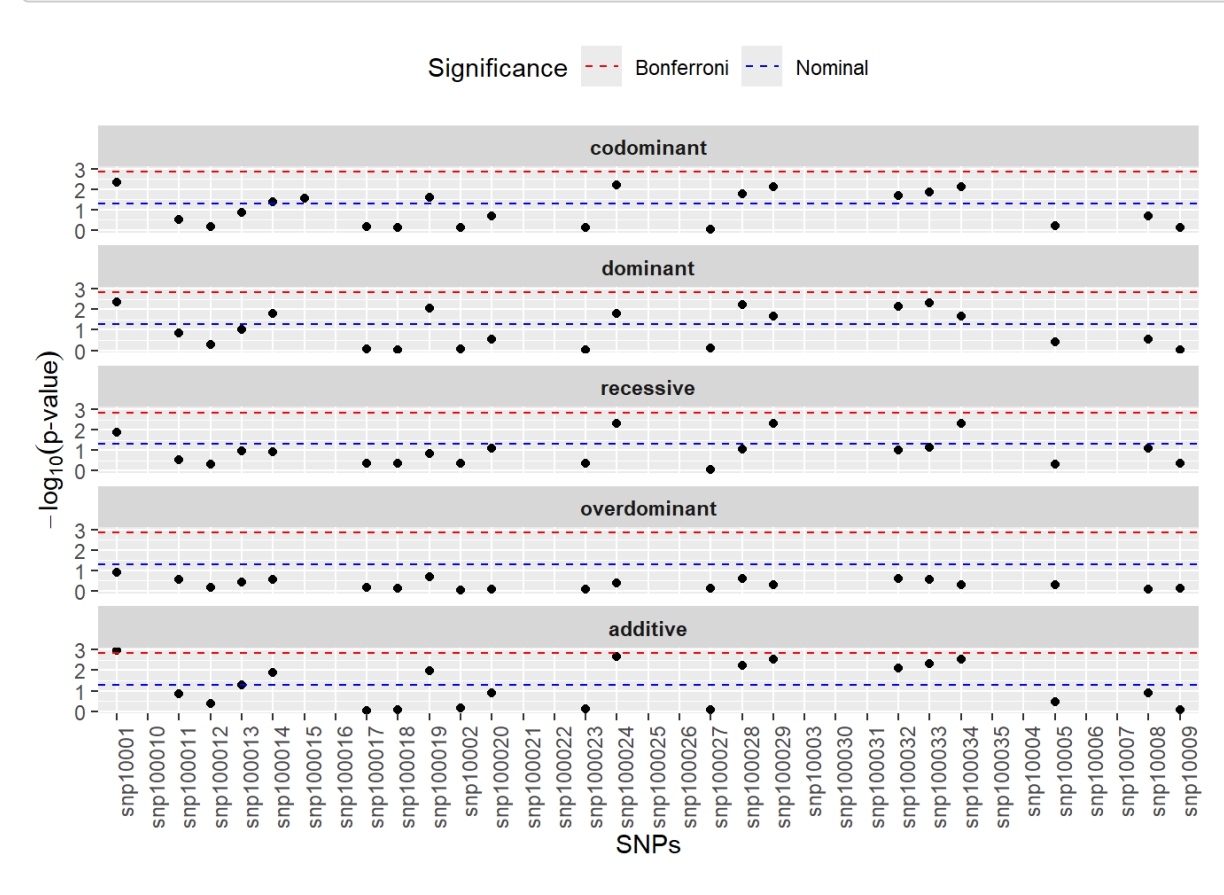
```
##           comments    codominant dominant recessive overdominant log-additive
## snp10001  -           0.00455    0.00434  0.01369   0.11976      0.00107
## snp10002  -           0.75803    0.84774  0.45893   0.94202      0.68014
## snp10003  Monomorphic -          -        -         -            -
## snp10004  Monomorphic -          -        -         -            -
## snp10005  -           0.59132    0.39483  0.49409   0.50915      0.33215
## snp10006  Monomorphic -          -        -         -            -
## snp10007  Monomorphic -          -        -         -            -
## snp10008  -           0.19743    0.27639  0.08442   0.79730      0.12365
## snp10009  -           0.74139    0.93626  0.45253   0.73010      0.86450
## snp100010 Monomorphic -          -        -         -            -
## snp100011 -           0.31979    0.13567  0.29144   0.28443      0.13669
## snp100012 -           0.65939    0.51995  0.46411   0.66092      0.43014
## snp100013 -           0.14356    0.09719  0.10392   0.38583      0.05264
## snp100014 -           0.04094    0.01640  0.12293   0.28145      0.01349
## snp100015 -           0.02814    -        -         -            -
## snp100016 Monomorphic -          -        -         -            -
## snp100017 -           0.69521    0.85979  0.42249   0.64732      0.92084
## snp100018 -           0.74239    0.93945  0.45253   0.73414      0.86217
## snp100019 -           0.02620    0.00828  0.14896   0.20175      0.01105
## snp100020 -           0.20049    0.28580  0.08442   0.81828      0.12794
## snp100021 Monomorphic -          -        -         -            -
## snp100022 Monomorphic -          -        -         -            -
## snp100023 -           0.75809    0.94203  0.45650   0.84703      0.75908
## snp100024 -           0.00602    0.01628  0.00472   0.42204      0.00222
## snp100025 Monomorphic -          -        -         -            -
## snp100026 Monomorphic -          -        -         -            -
## snp100027 -           0.94495    0.76735  0.90707   0.73867      0.81880
## snp100028 -           0.01653    0.00568  0.08925   0.24877      0.00605
## snp100029 -           0.00729    0.02051  0.00472   0.47760      0.00289
## snp100030 Monomorphic -          -        -         -            -
## snp100031 Genot 65%   -          -        -         -            -
## snp100032 -           0.02037    0.00684  0.09843   0.24337      0.00758
## snp100033 -           0.01361    0.00494  0.07077   0.27272      0.00468
## snp100034 -           0.00729    0.02051  0.00472   0.47760      0.00289
## snp100035 Monomorphic -          -        -         -            -
```

---

4. Los objetos de la clase `WGassociation` tienen implementado el método `plot()` que permite evaluar gráficamente el resultado. Interpreta el gráfico que se obtiene para la relación con la variable `protein`.

Solución:

```
plot(myres2)
```



Notad que tenemos marcados dos umbrales: el nivel de significación fijado, llamado nominal (por defecto 0.05), y el ajustado utilizando la corrección de Bonferroni. Los p-valores se representan en escala logarítmica, de forma que valores mayores se corresponden con modelos más significativos.

---

5. Podemos utilizar el estadístico máximo (González et al. 2008) para evaluar la asociación global, considerando simultáneamente los modelos de herencia dominante, recesivo y aditivo. Este estadístico está implementado para tablas 2x3 en la función `SNPassoc::maxstat()`, que requiere de dos argumentos:

- `x`, un objeto de la clase `setupSNP`
- `y`, el nombre de la variable del objeto `x` que contiene la información del estado caso / control.

Interpreta los resultados que se obtienen en este ejemplo con la sintaxis,

```
ans.max <- maxstat(mySNP, casco)
ans.max
```

Solución:

```
##           dominant recessive log-additive MAX-statistic Pr(>z)
## snp10001     1.559     1.209        0.286         1.559  0.384
## snp10002     0.012     1.479        0.068         1.479  0.411
## snp10003         -         -            -             -      -
## snp10004         -         -            -             -      -
## snp10005     0.013     0.491        0.005         0.491  0.740
## snp10006         -         -            -             -      -
## snp10007         -         -            -             -      -
## snp10008     0.173     0.286        0.017         0.286  0.832
## snp10009     0.019     1.429        0.053         1.429  0.424
## snp100010        -         -            -             -      -
## snp100011    0.258     0.007        0.147         0.258  0.818
## snp100012    0.022     0.444        0.094         0.444  0.761
## snp100013    0.620     0.425        0.126         0.620  0.670
## snp100014    0.588     0.327        0.043         0.588  0.685
## snp100015    0.005     0.005        0.005         0.005  0.946
## snp100016        -         -            -             -      -
## snp100017    0.046     1.379        0.026         1.379  0.437
## snp100018    0.103     1.429        0.005         1.429  0.424
## snp100019    1.264     0.370        0.155         1.264  0.450
## snp100020    0.279     0.286        0.047         0.286  0.832
## snp100021        -         -            -             -      -
## snp100022        -         -            -             -      -
## snp100023    0.017     1.473        0.060         1.473  0.413
## snp100024    1.012     0.508        0.209         1.012  0.529
## snp100025        -         -            -             -      -
## snp100026        -         -            -             -      -
## snp100027    0.223     1.451        0.004         1.451  0.420
## snp100028    1.186     0.540        0.080         1.186  0.470
## snp100029    0.378     0.508        0.025         0.508  0.720
## snp100030        -         -            -             -      -
## snp100031        -         -            -             -      -
## snp100032    0.737     0.540        0.019         0.737  0.617
## snp100033    0.885     0.664        0.018         0.885  0.564
## snp100034    0.378     0.508        0.025         0.508  0.719
## snp100035        -         -            -             -      -
```

Se obtiene un objeto de la clase `maxstat` que contiene la tabla de estadísticos test asociados a los modelos dominante, recesivo y log-aditivo, más dos columnas: el estadístico test máximo y su p-valor.

Equilibrio de Hardy-Weinberg (HWE)

En una población formada por un elevado número de individuos, con reproducción aleatoria entre ellos y sin que actúe ninguna fuerza evolutiva, las frecuencias de los alelos de un gen se mantienen estables, generación tras generación.

En los GWAS el HWE se utiliza como medida de calidad de los SNPs a estudiar. La prueba consiste en comparar las frecuencias genotípicas observadas con las esperadas según la ley de HW, es decir apareamiento aleatorio en ausencia de selección, mutación, deriva genética u otras fuerzas. **El HWE debe verificarse en los controles**.

En el paquete `SNPassoc` podemos comprobar el HWE utilizano el test exacto de Wigginton, Cutler y Abecasis (Wigginton, Cutler, and Abecasis 2005) implementado en la función `tableHWE()`. El primer argumento es el objeto de la clase `snp` y el segundo un `factor` en el que se almacena la información del estado caso/control.

1. Evalúa esta asunción en el ejemplo que estamos analizando. *Nota:* Puedes utilizar la función `as.factor(x)` para convertir el vector `x` en un `factor`.

Solución:

```
hwe <- tableHWE(mySNP,as.factor(mySNP$casco))
head(hwe)
```

```
##           all groups          0          1
## snp10001 0.281639248 0.46728042 0.06022047
## snp10002 0.004944837 0.02030711 0.09978450
## snp10003          NA         NA         NA
## snp10004          NA         NA         NA
## snp10005 0.008019904 0.04801125 0.07551332
## snp10006          NA         NA         NA
```

Se obtienen los p-valores asociados al contraste de HWE en la muestra completa (`all groups`) y en los grupos caso (`0`) y control (`1`) por separado.

---

2. Selecciona los SNPs que no están en equilibrio en el grupo de controles a nivel $\alpha = 0.05$. *Nota:* Puedes obtener un vector con los nombres de fila de una matriz con la función `rownames()`.

Solución:

```
rownames(hwe)[hwe[,2]<0.05 & !is.na(hwe[,2])]
```

```
## [1] "snp10002"  "snp10005"  "snp10009"  "snp100017" "snp100018" "snp100023"
## [7] "snp100027"
```

Interesa mantener en el análisis aquellos SNP que estan en equilibrio, es decir los que no rechazan la hipótesis nula en este contraste. Como se hacen múltiples contrastes podemos utilizar la corrección de Bonferroni. En este caso, como tenemos 35 SNPs, el nivel de significación individual a fijar para garantizar un nivel de significación simultáneo de 0.05, sería $\alpha = \frac{0.05}{35} = 0.0014$.

3. Utilizar la función `setupSNP()` eliminando de futuros análisis aquellos SNPs que no están en HWE en el grupo de controles. ¿Cuál es la dimensión del nuevo objeto `setupSNP`? ¿Cuántos SNPs han sido eliminados?

Solución:

```
snps.ok <- rownames(hwe)[hwe[,2]>=0.0014 | is.na(hwe[,2])]
pos <- which(colnames(SNPs)%in%snps.ok, useNames = FALSE)
mySNP.hwe <- setupSNP(SNPs, pos, sep="")
head(mySNP.hwe)
```

```
##   id casco    sex blood.pre  protein snp10001 snp10002 snp10003 snp10004
## 1  1     1 Female      13.7 75640.52      T/T      C/C      G/G      G/G
## 2  2     1 Female      12.7 28688.22      T/T      A/C      G/G      G/G
## 3  3     1 Female      12.9 17279.59      T/T      C/C      G/G      G/G
## 4  4     1   Male      14.6 27253.99      C/T      C/C      G/G      G/G
## 5  5     1 Female      13.4 38066.57      T/T      A/C      G/G      G/G
## 6  6     1 Female      11.3  9872.46      T/T      C/C      G/G      G/G
##   snp10005 snp10006 snp10007 snp10008 snp10009 snp100010 snp100011 snp100012
## 1      G/G      A/A      C/C      C/C      A/A       T/T       G/G       G/G
## 2      A/G      A/A      C/C      C/C      A/G       T/T       G/G       C/G
## 3      G/G      A/A      C/C      C/C      A/A       T/T       C/C       G/G
## 4      G/G      A/A      C/C      C/C      A/A       T/T       G/G       G/G
## 5      G/G      A/A      C/C      C/C      A/G       T/T       G/G       G/G
## 6      G/G      A/A      C/C      C/C      A/A       T/T       G/G       G/G
##   snp100013 snp100014 snp100015 snp100016 snp100017 snp100018 snp100019
## 1       A/A       A/A       G/G       G/G       T/T       T/T       C/C
## 2       A/A       A/C       G/G       G/G       C/T       C/T       C/G
## 3       A/A       C/C       G/G       G/G       T/T       T/T       C/C
## 4       A/A       A/C       G/G       G/G       T/T       T/T       C/G
## 5       A/A       A/C       G/G       G/G       C/T       C/T       C/G
## 6       A/A       A/A       G/G       G/G       T/T       T/T       C/C
##   snp100020 snp100021 snp100022 snp100023 snp100024 snp100025 snp100026
## 1       G/G       G/G       A/A       T/T       T/T       C/C       G/G
## 2       G/G       G/G       A/A       A/T       T/T       C/C       G/G
## 3       G/G       G/G       A/A       T/T       T/T       C/C       G/G
## 4       G/G       G/G       A/A       T/T       C/T       C/C       G/G
## 5       G/G       G/G       A/A       A/T       T/T       C/C       G/G
## 6       G/G       G/G       A/A       T/T       T/T       C/C       G/G
##   snp100027 snp100028 snp100029 snp100030 snp100031 snp100032 snp100033
## 1       C/C       C/C       G/G       A/A       T/T       A/A       A/A
## 2       C/G       C/T       G/G       A/A       T/T       A/G       A/G
## 3       C/C       C/C       G/G       A/A       T/T       A/A       A/A
## 4       C/C       C/T       A/G       A/A       T/T       A/G       A/G
## 5       C/G       C/T       G/G       A/A       T/T       A/G       A/G
## 6       C/C       C/C       G/G       A/A       T/T       A/A       A/A
##   snp100034 snp100035
## 1       T/T       T/T
## 2       T/T       T/T
## 3       T/T       T/T
## 4       C/T       T/T
## 5       T/T       T/T
## 6       T/T      <NA>
```

En este ejemplo no eliminamos ninguno de los SNPs. El nuevo objeto de la clase `setupSNP` tiene las mismas columnas que el antiguo,

```
dim(mySNP.hwe)
```

```
## [1] 157  40
```

```
dim(mySNP)
```

```
## [1] 157  40
```

Valores missings en los genotipos

Los objetos de la clase `snp` tienen implementados un método `summary()` con el que podemos obtener información básica sobre cada SNP analizado. Concretamente se muestra la frecuencia del alelo con mayor frecuencia, el p-valor para el test HWE y una tasa de valores *missings*.

1. Comprueba la tasa de valores *missings* en el ejemplo. ¿Qué SNP es el que tiene una mayor tasa de *missings*?

Solución:

El método summary() devuelve un data.frame que almacenamos en un nuevo objeto,

```
sum.snp<-summary(mySNP)
```

```
##           alleles major.allele.freq HWE      missing (%)
## snp10001  T/C      75.5             0.281639  0.0
## snp10002  C/A      72.0             0.004945  0.0
## snp10003  G       100.0             -         8.3
## snp10004  G       100.0             -         0.6
## snp10005  G/A      75.8             0.008020  0.0
## snp10006  A       100.0             -         0.0
## snp10007  C       100.0             -         0.0
## snp10008  C/G      80.3             0.137802  0.0
## snp10009  A/G      71.5             0.002848  0.6
## snp100010 T       100.0             -         6.4
## snp100011 G/C      98.7             0.019139  0.0
## snp100012 G/C      76.1             0.013399  1.3
## snp100013 A/G      81.7             0.025588  7.6
## snp100014 C/A      58.2             1.000000  2.5
## snp100015 G/A      95.9             1.000000  0.0
## snp100016 G       100.0             -         3.2
## snp100017 T/C      70.0             0.000518  1.3
## snp100018 T/C      69.9             0.000498  0.6
## snp100019 G/C      55.7             0.746284  0.0
## snp100020 G/A      80.6             0.125355  0.0
## snp100021 G       100.0             -         0.0
## snp100022 A       100.0             -         0.6
## snp100023 T/A      71.4             0.002842  1.9
## snp100024 T/C      74.7             0.092210  0.6
## snp100025 C       100.0             -         0.0
## snp100026 G       100.0             -         0.6
## snp100027 C/G      70.3             0.000896  1.3
## snp100028 T/C      55.1             0.419687  0.6
## snp100029 G/A      75.6             0.048709  0.6
## snp100030 A       100.0             -         0.0
## snp100031 T       100.0             -        35.0
## snp100032 G/A      55.8             0.258909  0.6
## snp100033 G/A      54.9             0.326373  3.2
## snp100034 T/C      75.6             0.048709  0.6
## snp100035 T       100.0             -         7.0
```

El SNP con un mayor tasa de valores perdidos es,

```
rownames(sum.snp)[which.max(sum.snp[,4])]
```

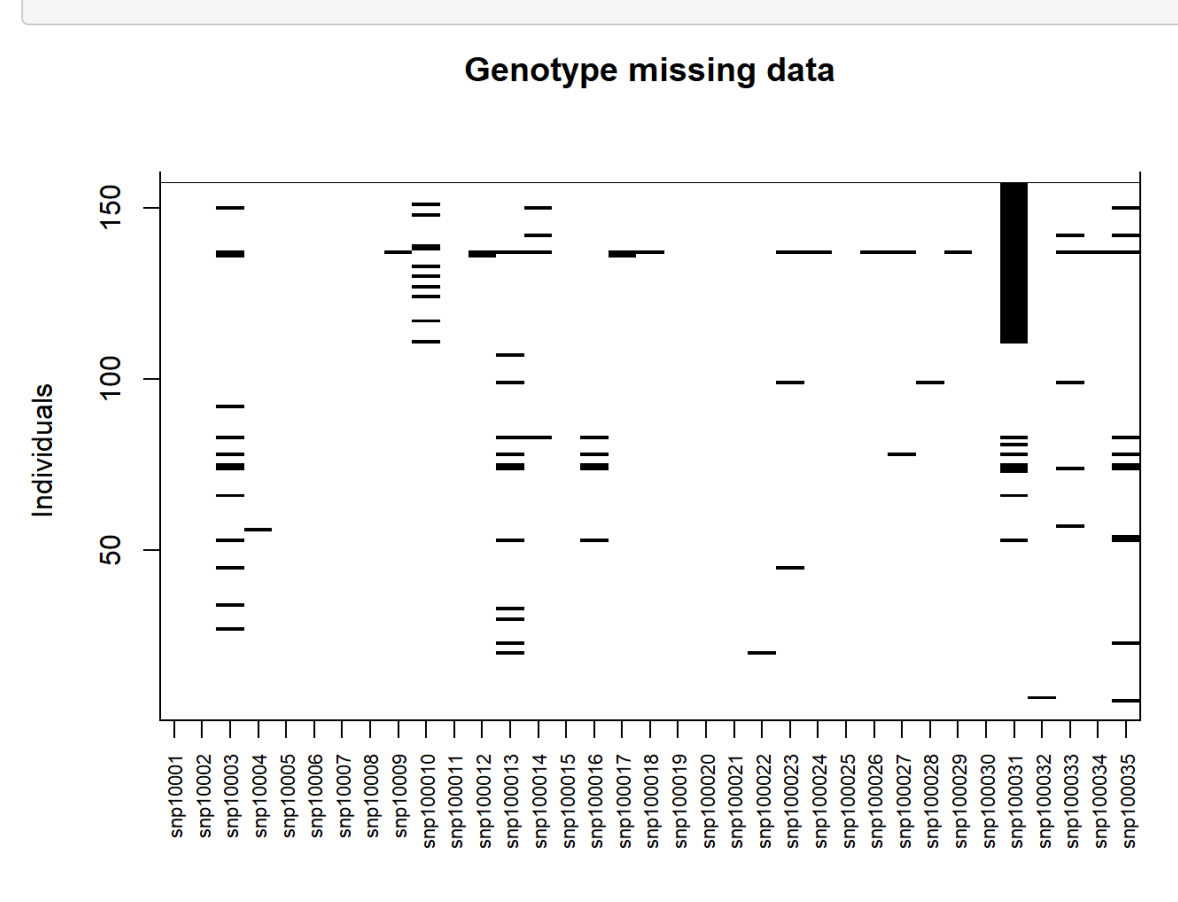
```
## [1] "snp100031"
```

---

2. Explora gráficamente el patrón de aparición de *missings* con la función `plotMissing()`.

Solución:

```
plotMissing(mySNP)
```



En este gráfico se muestra un esquema de los individuos que son *missing* en cada SNP evaluado. Es importante que los valores *missing* aparezcan distribuidos aleatoriamente entre individuos y SNPs, puesto que de otra forma podrían sesgar nuestros resultados. En este caso por ejemplo, el SNP 31 muestra un patrón claro de valores *missing* en los individuos, con una tasa mucho mayor entre los controles. En principio, en los análisis de asociación, no se tienen en cuenta los valores no observados, por lo que habría que tener precaución con las conclusiones a las que llegamos respecto a este marcador.

Podemos conocer exactamente al frecuencia de apación de *missings* en cada grupo construyendo una tabla de contingencia.

```
table(is.na(mySNP$snp100031),mySNP$casco)
```

```
##
##           0   1
##   FALSE   0 102
##   TRUE   47   8
```

Incluir interacciones gen-entorno y gen-gen

La función `association()` del paquete `SNPassoc` permite incluir en nuestros modelos **interacciones gen-entorno**.

1. Evalúa si el riesgo del SNP ‘snp10001’ es diferente en hombres que en mujeres considerando el modelo de herencia dominante. *Nota:* las interacciones en las fórmulas de R se incluyen con el operador `*`.

Solución:

```
association(casco ~ dominant(snp10001)*factor(sex), data=mySNP)
```

```
##
##       SNP: dominant(snp10001  adjusted by:
##  Interaction
## ---------------------
##         Male      OR lower upper Female      OR lower upper
## T/T       11 29 1.00    NA    NA     13 39 1.14  0.45  2.90
## C/T-C/C   10 25 0.95  0.35   2.6     13 17 0.50  0.18  1.35
##
## p interaction: 0.27301
##
##  factor(sex) within dominant(snp10001
## ---------------------
## T/T
##         0  1   OR lower upper
## Male   11 29 1.00    NA    NA
## Female 13 39 1.14  0.45   2.9
##
## C/T-C/C
##         0  1   OR lower upper
## Male   10 25 1.00    NA    NA
## Female 13 17 0.52  0.19  1.46
##
## p trend: 0.27301
##
##  dominant(snp10001 within factor(sex)
## ---------------------
## Male
##          0  1   OR lower upper
## T/T     11 29 1.00    NA    NA
## C/T-C/C 10 25 0.95  0.35   2.6
##
## Female
##          0  1   OR lower upper
## T/T     13 39 1.00    NA    NA
## C/T-C/C 13 17 0.44  0.17  1.13
##
## p trend: 0.27301
```

El resultado muestra que portar el alelo C decrementa el riesgo más en las mujeres (OR=0.50; IC: 0.18-1.35) que en los hombres (OR=0.95; IC: 0.35-2.6), aunque en ninguno de los dos casos es estadísticamente significativo (los IC contienen al 1). La interacción no es estadísticamente significativa (p-valor de la interaccion=0.27301).

---

2. Análogamente se pueden estudiar **interacciones gen-gen**. Estudiar la interacción entre los dos primeros SNPs.

Solución:

```
association(casco ~ snp10001*factor(snp10002), data=mySNP, model.interaction = "dominant")
```

```
##
##       SNP: snp10001  adjusted by:
##  Interaction
## ---------------------
##         C/C      OR lower upper A/C      OR lower upper 0 1 A/A lower upper
## T/T      13 25 1.00    NA    NA  11 38 1.80  0.70  4.64 0 5  NA     0    NA
## C/T-C/C   9 27 1.56  0.57  4.28  14 15 0.56  0.21  1.50 0 0  NA    NA    NA
##
## p interaction: 0.023877
##
##  factor(snp10002) within snp10001
## ---------------------
## T/T
##      0  1  OR lower upper
## C/C 13 25 1.0    NA    NA
## A/C 11 38 1.8   0.7  4.64
## A/A  0  5  NA   0.0    NA
##
## C/T-C/C
##      0  1   OR lower upper
## C/C  9 27 1.00    NA    NA
## A/C 14 15 0.36  0.13  1.02
## A/A  0  0   NA    NA    NA
##
## p trend: 0.023877
##
##  snp10001 within factor(snp10002)
## ---------------------
## C/C
##          0  1   OR lower upper
## T/T     13 25 1.00    NA    NA
## C/T-C/C  9 27 1.56  0.57  4.28
##
## A/C
##          0  1   OR lower upper
## T/T     11 38 1.00    NA    NA
## C/T-C/C 14 15 0.31  0.12  0.83
##
## A/A
##         0 1 OR lower upper
## T/T     0 5  1    NA    NA
## C/T-C/C 0 0 NA    NA    NA
##
## p trend: 0.0077424
```

Notad que uno de los términos de la interacción tiene que ser de tipo `factor`.

El resultado muestra que portar el alelo C en el ‘snp10001’ incrementa el riesgo en los individuos con genotipo CC en ‘snp10002’ (OR=1.56; IC: 0.57-4.28) mientras que lo decrementa en los AC (OR=0.56 IC: 0.21-1.5), aunque en ninguno de los dos casos es estadísticamente significativo (los IC contienen al 1). La interacción es estadísticamente significativa (p-valor de la interacción=0.0239).

Visualización de los resultados de un GWAS

Realizamos el GWAS con los datos de `SNPs` utilizando `protein` como variable dependiente.

```
library("SNPassoc")
data(SNPs)
datSNP<-setupSNP(SNPs,6:40,sep="")
myres <- WGassociation(protein,data=datSNP,model="all")
myres
```

```
##           comments    codominant dominant recessive overdominant log-additive
## snp10001  -           0.00586    0.00516  0.01605   0.12306      0.00140
## snp10002  -           0.78555    0.93303  0.48704   0.87288      0.76846
## snp10003  Monomorphic -          -        -         -            -
## snp10004  Monomorphic -          -        -         -            -
## snp10005  -           0.63306    0.43881  0.50130   0.55427      0.37267
## snp10006  Monomorphic -          -        -         -            -
## snp10007  Monomorphic -          -        -         -            -
## snp10008  -           0.20627    0.30005  0.08652   0.83655      0.13493
## snp10009  -           0.74736    0.87204  0.47815   0.68152      0.93616
## snp100010 Monomorphic -          -        -         -            -
## snp100011 -           0.30538    0.12921  0.27289   0.28415      0.12787
## snp100012 -           0.70573    0.58360  0.47929   0.72341      0.48994
## snp100013 -           0.14971    0.09878  0.11040   0.38420      0.05475
## snp100014 -           0.03796    0.01511  0.11952   0.27139      0.01245
## snp100015 -           0.02626    -        -         -            -
## snp100016 Monomorphic -          -        -         -            -
## snp100017 -           0.70644    0.79126  0.45966   0.59972      0.99917
## snp100018 -           0.75290    0.88692  0.47815   0.69529      0.92129
## snp100019 -           0.02401    0.00749  0.14775   0.19168      0.01024
## snp100020 -           0.21002    0.31381  0.08652   0.86338      0.14135
## snp100021 Monomorphic -          -        -         -            -
## snp100022 Monomorphic -          -        -         -            -
## snp100023 -           0.77537    0.99544  0.48195   0.79887      0.82695
## snp100024 -           0.00724    0.01774  0.00548   0.42442      0.00263
## snp100025 Monomorphic -          -        -         -            -
## snp100026 Monomorphic -          -        -         -            -
## snp100027 -           0.92849    0.71496  0.94900   0.69971      0.75864
## snp100028 -           0.01480    0.00504  0.08758   0.23535      0.00545
## snp100029 -           0.00859    0.02183  0.00548   0.47601      0.00333
## snp100030 Monomorphic -          -        -         -            -
## snp100031 Genot 65%   -          -        -         -            -
## snp100032 -           0.01943    0.00656  0.09656   0.23993      0.00717
## snp100033 -           0.01253    0.00455  0.06839   0.26544      0.00428
## snp100034 -           0.00859    0.02183  0.00548   0.47601      0.00333
## snp100035 Monomorphic -          -        -         -            -
```

Para presentar los resultados de un GWAS podemos utilizar diferentes tipos de gráficos. Necesitamos el paquete `GWASTools` disponible en Bioconductor.

0. Instala y carga el paquete `GWASTools` disponible en Bioconductor.

Solución:

```
if(!("GWASTools" %in% installed.packages()))
  BiocManager::install("GWASTools")
library("GWASTools")
```

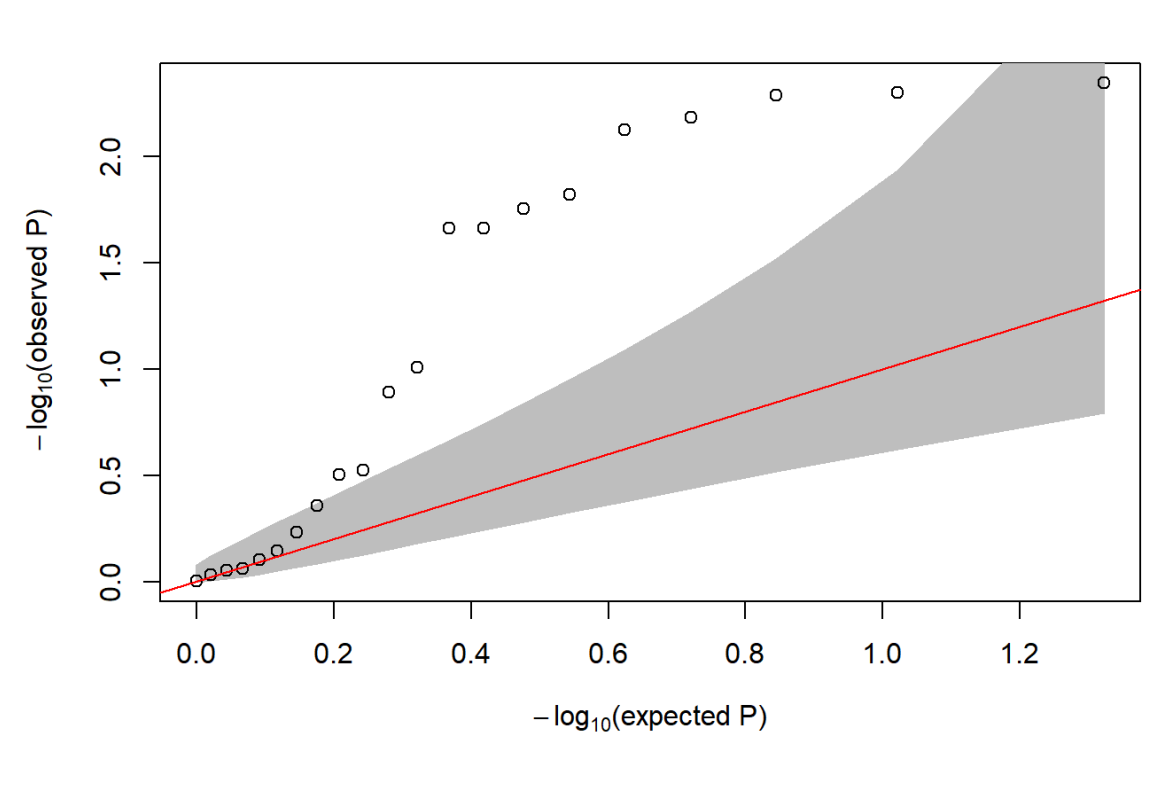
---

1. El primer gráfico que vamos a construir es un **QQ-plot**. En general, este gráfico se representa la distribución esperada en el eje X contra la observada en el eje Y. Desviaciones de la recta $Y = X$ implican que existen diferencias importantes entre los casos y los controles.

El **QQ-plot** se construye con la función `qqPlot()` del paquete `GWASTools`. Como argumento requiere un vector de p-valores. Construye el **QQ-plot** asociado al modelo dominante.

Solución:

```
pvals <- dominant(myres)
qqPlot(pvals)
```



---

2. Otro gráfico interesante es el **Manhattan plot**. Este plot es útil para visualizar resultados en un conjunto grande de SNPs. El `data.frame` `SNPs` es pequeño, con sólo 35 SNPs.

2.1. Simula unos resultados sobre un conjunto de 1000 SNPs. Debes simular 1000 p-valores aleatorios de los 1000 contrastes de asociación. Utilizamos la función `sample()`.

Solución:

```
set.seed(25683)
n <- 1000
pvals <- sample(-log10((1:n)/n), n, replace=TRUE)
```

2.2. Asigna esos p-valores aleatoriamente a los diferentes cromosomas.

Solución:

Por ejemplo, asigno 100 al cromosoma 1, 150 al 2, 80 al 3, 90 al 4…

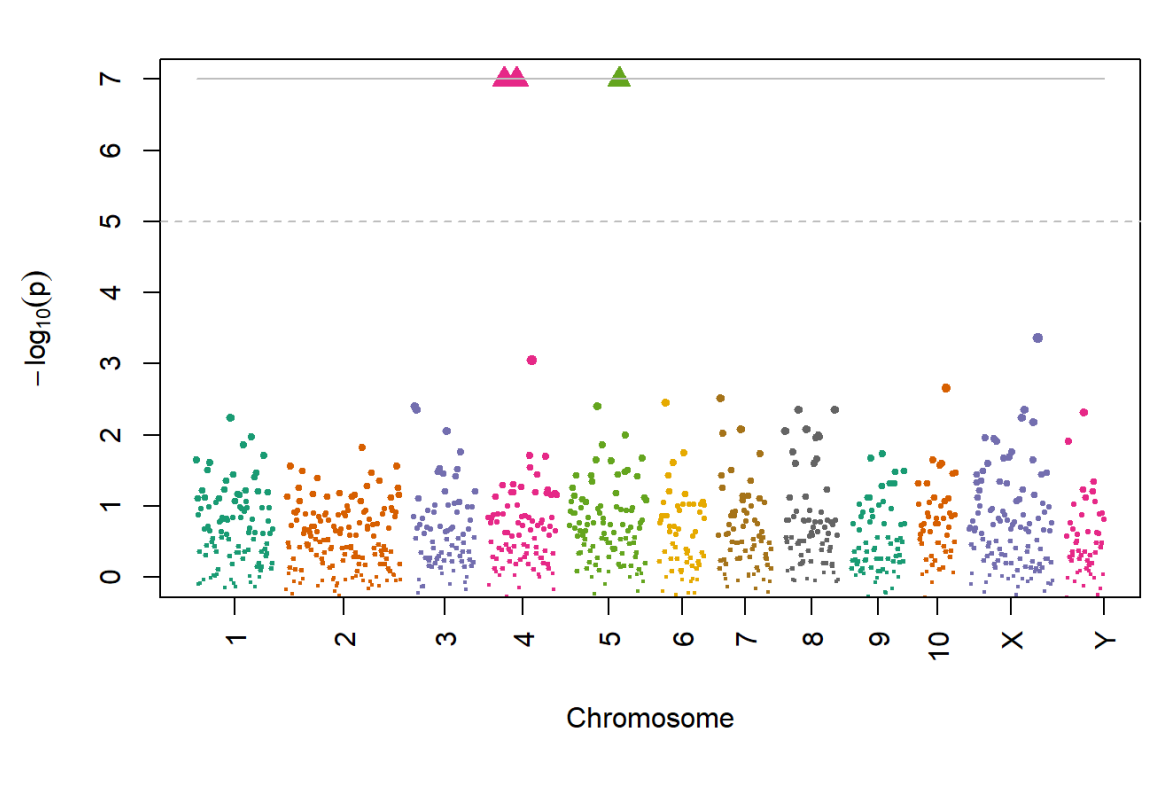
```
chromosome <- c(rep(1,100), rep(2,150), rep(3, 80),rep(4,90), rep(5,100), rep(6,60),
                rep(7, 70), rep(8, 70), rep(9,70),rep(10,50),rep("X",110), rep("Y",50))
length(chromosome)
```

```
## [1] 1000
```

2.3. Construye el plot con la función `manhattanPlot()`. Esta función requiere un primer argumento con los p-valores, el segundo con la asignación al cromosoma, y podéis especificar el nivel de significación con el argumento `signif`. El valor por defecto para este umbral es `5e-8`.

Solución: Establecemos un nivel de significación con el argumento `signif`.

```
manhattanPlot(pvals, chromosome,signif=1e-5)
```



Cada p-valor se representa con un punto cuyo tamaño es mayor cuanto más significativo es el contraste de asociación.

Referencias

González, Juan R, Josep L Carrasco, Frank Dudbridge, Lluı̈s Armengol, Xavier Estivill, and Victor Moreno. 2008. “Maximizing Association Statistics over Genetic Models.” *Genetic Epidemiology: The Official Publication of the International Genetic Epidemiology Society* 32 (3): 246–54.

Wigginton, Janis E, David J Cutler, and Gonçalo R Abecasis. 2005. “A Note on Exact Tests of Hardy-Weinberg Equilibrium.” *The American Journal of Human Genetics* 76 (5): 887–93.